

Módulo 0 - Guía práctica

Curso Spring Boot 2026

Índice

Módulo 0 - Guía práctica: del entorno vacío a la primera aplicación Spring Boot ejecutable	5
Qué vas a construir	5
Práctica 0.1 - Verificar el entorno y ejecutar Java sin Spring Boot	6
Objetivo	6
Paso 1 - Abrir una terminal adecuada	6
Windows	6
Linux	6
macOS	6
Cómo elegir la terminal correcta en cada sistema	6
Qué significa el prompt	6
Error frecuente: ejecutar como administrador “por si acaso”	7
Qué estamos verificando	7
Errores habituales	7
Pregunta	7
Respuesta razonada	7
Paso 2 - Verificar el runtime Java	7
Qué hace	7
Qué debes observar	7
Interpretar la salida con más precisión	8
Localizar el ejecutable real	8
Si aparece “command not found” o “is not recognized”	8
Si aparece Java 8 u 11	8
Pregunta	8
Respuesta razonada	9
Paso 3 - Verificar el compilador	9
Qué hace javac	9
Qué debes comparar	9
Diagnóstico útil	9
Pregunta	9
Respuesta razonada	9
Paso 4 - Verificar Maven	9
Interpretar mvn -version línea por línea	10
Distinguir Maven home de Java home	10
Primera ejecución lenta	10
La línea más importante para el diagnóstico	10
Si mvn no existe	10
Pregunta	11
Respuesta razonada	11
Paso 5 - Verificar JAVA_HOME	11
PowerShell	11
CMD	11
Linux/macOS	11
Comprueba también PATH	11
Pregunta	11

Respuesta razonada	11
Paso 6 - Crear el programa Java mínimo	12
Explicación línea por línea	12
Pregunta	13
Respuesta razonada	13
Paso 7 - Compilar el programa	13
Qué ha ocurrido	13
Errores frecuentes	13
Pregunta	13
Respuesta razonada	13
Paso 8 - Ejecutar el programa	14
Desglose	14
Errores frecuentes	14
Pregunta	14
Respuesta razonada	14
Paso 9 - Confirmar nuevamente Maven	14
Paso 10 - Configurar el IDE	14
IntelliJ IDEA	14
Eclipse	15
VS Code	15
Verificación cruzada	15
Errores frecuentes	15
Pregunta	15
Respuesta razonada	16
Paso 11 - Tabla de diagnóstico del entorno	16
Paso 12 - Reto resuelto: argumento de línea de comandos	16
Qué hemos añadido	16
Pregunta	16
Respuesta razonada	17
Resultado esperado de la práctica 0.1	17
Práctica 0.2 - Crear el proyecto Spring Boot	17
Paso 1 - Abrir Spring Initializr	17
Pregunta	17
Respuesta razonada	17
Paso 2 - Configurar Project, Language y Spring Boot	17
Valores exactos de referencia para el curso	17
Por qué estos nombres importan	18
Pregunta	18
Respuesta razonada	18
Paso 3 - Rellenar Project Metadata	18
Qué significa cada campo	18
Pregunta	19
Respuesta razonada	19
Paso 4 - Añadir dependencias	19
Pregunta	19
Respuesta razonada	19
Paso 5 - Generar y descargar	19
Paso 6 - Descomprimir y reconocer la raíz	19
Error común	19
Paso 7 - Abrir el proyecto en el IDE	19
IntelliJ	19
Eclipse	20
VS Code	20
IntelliJ IDEA: importación detallada	20
Eclipse: importación detallada	20
VS Code: importación detallada	20
Verificación independiente del IDE	20

Paso 8 - Verificar JDK y modelo Maven	20
Paso 9 - Ejecutar desde el IDE	21
Pregunta	21
Respuesta razonada	21
Paso 10 - Probar la raíz	21
Pregunta	21
Respuesta razonada	21
Paso 11 - Diagnóstico de creación/importación	21
Paso 12 - Reto resuelto: añadir DevTools manualmente	21
Explicación	22
Pregunta	22
Respuesta razonada	22
Variante adicional - Crear el mismo proyecto desde IntelliJ, Eclipse, VS Code y consola	22
IntelliJ IDEA	22
Eclipse + Spring Tools	23
VS Code	23
Consola mediante Spring Initializr	23
Verificación común de las cuatro vías	24
Pregunta	24
Respuesta razonada	24
Resultado esperado de la práctica 0.2	24
Práctica 0.3 - Entender la estructura y crear los primeros endpoints	24
Paso 1 - Abrir la raíz del proyecto	24
Paso 2 - Explorar la estructura completa	24
Recorre cada elemento y clasificalo	25
Experimento controlado con target	25
Inventario razonado de la raíz	25
Pregunta	26
Respuesta razonada	26
Paso 3 - Analizar pom.xml	26
No leas el POM como XML genérico: léelo por responsabilidades	26
Pregunta	28
Respuesta razonada	28
Paso 4 - Analizar la clase principal	28
Qué debes entender	28
Pregunta	28
Respuesta razonada	28
Paso 5 - Analizar application.properties	28
Pregunta	29
Respuesta razonada	29
Paso 6 - Crear el paquete controller	29
Paso 7 - Crear SaludoController	29
Lee el archivo de arriba abajo	30
Qué hace @RestController	30
Qué hace @GetMapping	30
Qué no hemos añadido todavía	30
Explicación completa	30
Errores frecuentes	30
Pregunta	31
Respuesta razonada	31
Paso 8 - Arrancar la aplicación	31
Paso 9 - Probar /hola en navegador	31
Paso 10 - Probar con curl	31
Pregunta	31
Respuesta razonada	31
Paso 11 - Diagnóstico de estructura/endpoints	32
Paso 12 - Reto resuelto: consolidar /adios	32

Pregunta	32
Respuesta razonada	32
Variante adicional - Crear paquetes y clases en los cuatro entornos	32
Desde consola y editor de texto	32
IntelliJ IDEA	33
Eclipse	33
VS Code	33
Por qué enseñamos también la consola	33
Pregunta	33
Respuesta razonada	34
Resultado esperado de la práctica 0.3	34
Práctica 0.4 - Ciclo de trabajo diario sin depender de módulos posteriores	34
Requisitos previos	34
Paso 1 - Arrancar desde el IDE	34
IntelliJ IDEA	34
Eclipse	34
VS Code	34
IntelliJ IDEA: ejecución y configuración	34
Eclipse: ejecución y configuración	35
VS Code: ejecución y configuración	35
Verificación funcional común	35
Qué debes anotar	35
Pregunta	35
Respuesta razonada	35
Paso 2 - Leer los logs con intención	36
Experimento temporal	36
Pregunta	36
Respuesta razonada	36
Paso 3 - Parar desde el IDE	36
Pregunta	36
Respuesta razonada	37
Paso 4 - Arrancar desde terminal	37
Pregunta	37
Respuesta razonada	37
Paso 5 - Poner un breakpoint	37
Colocar el breakpoint según el IDE	37
Qué ocurre cuando haces la petición	37
Qué observar	38
Pregunta	38
Respuesta razonada	38
Paso 6 - Comparar Step Over y Step Into	38
Pregunta	38
Respuesta razonada	38
Paso 7 - Crear y ejecutar un test del controlador	39
Ejecutar la misma prueba en cada IDE	39
Qué debes aprender de un fallo	39
Por qué este test es deliberadamente simple	40
Pregunta	40
Respuesta razonada	40
Paso 8 - Ejecutar todos los tests desde terminal	40
Pregunta	40
Respuesta razonada	40
Paso 9 - Probar la API con curl	40
Construye una pequeña matriz de observación	41
Qué debes observar en una respuesta	41
Prueba otro puerto sin modificar el código	41
Errores frecuentes	42

Pregunta	42
Respuesta razonada	42
Paso 10 - Enviar cabeceras y cookies para observar la petición	42
Pregunta	42
Respuesta razonada	42
Paso 11 - Tabla de errores del ciclo diario	42
Diagnóstico antes de «probar cosas»	42
Paso 12 - Reto resuelto: localizar un fallo con el depurador	43
Estado temporal: introduce un defecto	43
Ejecuta sólo el test afectado	43
Depura el test	43
Estado final obligatorio	44
Pregunta	44
Respuesta razonada	44
Recorridos completos por entorno: consola, IntelliJ IDEA, Eclipse y VS Code	44
Recorrido A - Ciclo diario exclusivamente desde consola	44
Recorrido B - Ciclo diario completo en IntelliJ IDEA	47
Recorrido C - Ciclo diario completo en Eclipse	49
Recorrido D - Ciclo diario completo en VS Code	51
Tabla de equivalencias rápida de los cuatro recorridos	53
Pregunta de integración	53
Respuesta razonada	53
Empaquetado y verificación final del Módulo 0	53
Paso final A - Ejecutar la suite	53
Paso final B - Empaquetar	54
Paso final C - Arrancar el JAR	54
Resultado esperado global del Módulo 0	54
Estado final de los ficheros que has construido	54
MiProyectoApplication.java	54
SaludoController.java	54
application.properties	55
SaludoControllerTest.java	55
Puente al Módulo 1	55

Módulo 0 - Guía práctica: del entorno vacío a la primera aplicación Spring Boot ejecutable

Qué vas a construir

En este módulo no vamos a utilizar un proyecto “mágico” preparado de antemano. Empezaremos verificando Java y Maven, compilaremos un programa Java sin Spring, crearemos un proyecto Spring Boot, estudiaremos su estructura y terminaremos con una aplicación web que:

- arranca desde IDE y terminal;
- expone GET /hola y GET /adios;
- contiene tests ejecutables;
- se empaqueta como JAR;
- se puede ejecutar con `java -jar`;
- permite practicar breakpoints, Step Over y Step Into sin depender de contenidos de módulos posteriores.

El proyecto final del módulo está bajo `M0/proyecto/`. El pequeño programa Java previo está bajo `M0/ejemplos/HolaMinisterio.java`.

Baseline del curso: Java 17 como nivel de lenguaje, Maven 3.9.x y Spring Boot 3.5.16.

Práctica 0.1 - Verificar el entorno y ejecutar Java sin Spring Boot

Objetivo

Antes de atribuir un error a Spring necesitamos demostrar que la plataforma Java funciona por sí sola. Esta práctica termina con un programa compilado mediante `javac` y ejecutado mediante `java`.

Paso 1 - Abrir una terminal adecuada

Windows

Puedes utilizar **PowerShell** o **CMD**. Para esta primera verificación es preferible usar una terminal normal, no una consola que mantenga una configuración de entorno distinta.

PowerShell:

```
PS C:\Users\Alumno>
```

CMD:

```
C:\Users\Alumno>
```

Linux

Abre el terminal de tu distribución. En Ubuntu, normalmente puedes utilizar `Ctrl+Alt+T`.

macOS

Abre Terminal desde Spotlight o Aplicaciones > Utilidades.

Cómo elegir la terminal correcta en cada sistema

En Windows tienes varias opciones. Para diagnosticar el entorno del sistema, empieza por **PowerShell** o **CMD**. Git Bash y terminales incluidas por herramientas pueden tener su propio PATH, de modo que una prueba hecha allí no siempre representa la configuración que verá otra aplicación.

En Linux, una terminal gráfica suele arrancar un shell como Bash. En macOS es frecuente usar Zsh.

Antes de continuar, escribe un comando trivial de cada entorno:

PowerShell:

```
$PSVersionTable.PSVersion
```

CMD:

```
ver
```

Linux/macOS:

```
echo "$SHELL"
```

No necesitamos una versión concreta del shell. Sólo queremos saber qué intérprete está procesando nuestros comandos, porque la sintaxis de variables cambia entre PowerShell, CMD y Bash/Zsh.

Qué significa el prompt

Ejemplos:

```
PS C:\Users\Ana>
C:\Users\Ana>
ana@ubuntu:~$
ana@macbook ~ %
```

El prompt no forma parte del comando. Cuando la guía muestra:

```
java -version
```

debes escribir únicamente `java -version`.

Error frecuente: ejecutar como administrador “por si acaso”

Para verificar versiones, compilar y ejecutar el ejercicio Java no necesitas privilegios de administrador.

Trabajar siempre como administrador puede ocultar errores de permisos y crear archivos pertenecientes a otro usuario. Usa elevación sólo cuando la instalación o una operación del sistema realmente la requiera.

Qué estamos verificando

La terminal nos muestra el entorno que ve el sistema operativo. Un IDE puede tener configurada una JDK diferente; por eso todavía no queremos depender de él.

Errores habituales

Situación	Por qué puede confundir	Qué ha
Terminal abierta antes de cambiar PATH	Puede conservar variables antiguas	Cerrar
Git Bash en Windows resuelve otra ruta	Puede cargar configuración de shell propia	Contra
Terminal como administrador	Introduce privilegios innecesarios	Usar u
Estar en “otra carpeta”	Los comandos de versión no dependen del directorio actual	Ejecuta

Pregunta

¿Por qué no empezamos comprobando Java desde el IDE?

Respuesta razonada

Porque queremos separar dos niveles. Primero verificamos que el sistema puede localizar y ejecutar Java. Después verificaremos que el IDE utiliza una JDK coherente. Si empezamos por el IDE y algo falla, todavía no sabríamos si el problema está en Java o en la configuración del propio IDE.

Paso 2 - Verificar el runtime Java

Ejecuta:

```
java -version
```

Qué hace

El shell busca un ejecutable `java` en `PATH` y lo ejecuta con el argumento `-version`.

Qué debes observar

Necesitamos una JDK compatible con el proyecto. Para reproducir exactamente el curso, utiliza Java 17. Una salida válida tendrá una primera línea equivalente a:

```
openjdk version "17.x.x" ...
```

El proveedor puede ser Temurin, OpenJDK, Corretto, Oracle u otro compatible. La marca exacta no cambia los conceptos que vamos a estudiar.

Interpretar la salida con más precisión

Una salida de Temurin/OpenJDK puede incluir:

```
openjdk version "17.0.x" ...  
OpenJDK Runtime Environment ...  
OpenJDK 64-Bit Server VM ...
```

No todas las distribuciones imprimen exactamente las mismas palabras. Para este curso debes confirmar principalmente:

1. versión mayor 17;
2. arquitectura apropiada para tu equipo;
3. ausencia de errores de instalación;
4. coherencia posterior con javac y Maven.

Si aparece Java 21, el proyecto puede funcionar, pero no estarías reproduciendo exactamente el baseline pedagógico. Para evitar diferencias en ejemplos y bytecode, mantendremos Java 17 como JDK activa del curso.

Localizar el ejecutable real

PowerShell:

```
Get-Command java
```

CMD:

```
where java
```

Linux/macOS:

```
which java
```

Haz esta comprobación si el número de versión no coincide con lo esperado. Saber qué archivo se ejecuta suele ser más útil que reinstalar Java sin diagnóstico.

Si aparece “command not found” o “is not recognized”

No concluyas inmediatamente que Java no está instalado. Primero determina si existe una JDK y si PATH la incluye.

Windows:

```
where java
```

Linux/macOS:

```
which java
```

Si no aparece una ruta, revisa la instalación y PATH.

Si aparece Java 8 u 11

Probablemente hay varias instalaciones y la primera de PATH no es la que quieres usar. Localiza todas las rutas, corrige el orden o elimina referencias antiguas.

Pregunta

¿java -version demuestra que tenemos un compilador?

Respuesta razonada

No. Demuestra que podemos localizar un runtime Java. La herramienta que compila se llama javac; la comprobaremos de forma independiente.

Paso 3 - Verificar el compilador

Ejecuta:

```
javac -version
```

Resultado esperado:

```
javac 17.x.x
```

Qué hace javac

javac transforma ficheros .java en clases .class que contienen bytecode.

Qué debes comparar

Compara las dos versiones:

```
java -> 17.x
```

```
javac -> 17.x
```

No es buena señal obtener, por ejemplo:

```
java -> 11
```

```
javac -> 17
```

porque ejecución y compilación podrían estar resolviéndose desde instalaciones distintas.

Diagnóstico útil

Windows:

```
where java
```

```
where javac
```

Linux/macOS:

```
which java
```

```
which javac
```

Pregunta

¿Qué problema podría aparecer si compilas con Java 17 pero ejecutas el .class con una JVM 11?

Respuesta razonada

La JVM 11 puede rechazar un bytecode producido para una versión de clase posterior. El síntoma típico es un error de versión de clase, no un error de la lógica del programa. Por eso la coherencia entre compilador y runtime forma parte del diagnóstico.

Paso 4 - Verificar Maven

Ejecuta:

```
mvn -version
```

Si no tienes Maven globalmente, no pasa nada para el proyecto final porque utilizaremos Maven Wrapper; este paso sigue siendo útil para conocer tu entorno.

Una salida de Maven contiene, entre otros datos:

```
Apache Maven 3.9.x
Maven home: ...
Java version: 17.x.x
Java home: ...
OS name: ...
```

Interpretar mvn -version línea por línea

Una salida típica incluye información equivalente a:

```
Apache Maven 3.9.x
Maven home: ...
Java version: 17...
Java home: ...
Default locale: ...
OS name: ...
```

Comprueba dos versiones distintas:

- **Maven:** 3.9.x en nuestra referencia.
- **Java usada por Maven:** 17.

Es perfectamente posible que java -version muestre 17 y Maven indique otra versión si la configuración local está mezclada.

Distinguir Maven home de Java home

Maven home apunta a Maven.

Java home de la salida Maven indica la instalación Java que está usando Maven.

No deben apuntar a la misma carpeta. Son herramientas distintas.

Primera ejecución lenta

Si es la primera vez que Maven se ejecuta, puede descargar plugins y artefactos. Ver líneas de Downloading from central no indica un fallo.

Si la red está bloqueada, sí pueden aparecer timeouts o errores de resolución. En ese caso el problema no es el código Java: Maven no ha podido obtener lo que necesita.

La línea más importante para el diagnóstico

Fíjate en:

```
Java version: ...
Java home: ...
```

Maven puede estar utilizando una JDK distinta de la que obtienes al escribir directamente java -version.

Si mvn no existe

Tienes dos alternativas válidas:

1. instalar Maven 3.9.x globalmente;
2. continuar y utilizar mvnw/mvnw.cmd dentro del proyecto.

En este curso preferiremos el Wrapper para los comandos del proyecto.

Pregunta

Si java -version dice 17 pero mvn -version dice Java 11, ¿qué dato revisarías?

Respuesta razonada

Revisaría primero JAVA_HOME y la configuración con la que Maven selecciona Java. El shell puede encontrar java mediante PATH, mientras Maven puede arrancar utilizando una ruta configurada de forma diferente.

Paso 5 - Verificar JAVA_HOME

PowerShell

```
echo $env:JAVA_HOME
```

CMD

```
echo %JAVA_HOME%
```

Linux/macOS

```
echo "$JAVA_HOME"
```

Debe apuntar a la raíz de la JDK, no a bin.

Ejemplo conceptual:

C:\Program Files\Eclipse Adoptium\jdk-17...

No:

C:\Program Files\Eclipse Adoptium\jdk-17...\bin

Comprueba también PATH

Windows:

```
echo %PATH%
```

PowerShell:

```
echo $env:PATH
```

Linux/macOS:

```
echo "$PATH"
```

Pregunta

¿JAVA_HOME y PATH son dos nombres para lo mismo?

Respuesta razonada

No. JAVA_HOME suele contener una única ruta a la raíz de la JDK. PATH es una lista de directorios donde el shell busca ejecutables. Habitualmente se añade \$JAVA_HOME/bin o %JAVA_HOME%\bin a PATH, pero cumplen funciones distintas.

Paso 6 - Crear el programa Java mínimo

Crea una carpeta de trabajo y dentro una carpeta src.

Linux/macOS:

```
mkdir -p HolaMinisterio/src
cd HolaMinisterio
```

Windows CMD:

```
mkdir HolaMinisterio\src
cd HolaMinisterio
```

Crea src/HolaMinisterio.java con este primer estado:

```
public class HolaMinisterio {
    public static void main(String[] args) {
        System.out.println("Hola, Ministerio de Educación");
    }
}
```

Explicación línea por línea

public class HolaMinisterio {

- **public**: la clase es accesible públicamente.
- **class**: declara un tipo clase.
- **HolaMinisterio**: debe coincidir con el nombre del fichero público `HolaMinisterio.java`.
- **{**: abre el cuerpo de la clase.

Si el archivo se llamara `Saludo.java` pero la clase pública fuera `HolaMinisterio`, `javac` informaría de que la clase pública debe declararse en un archivo con el mismo nombre.

public static void main(String[] args) {

- **public**: la JVM debe poder acceder al método.
- **static**: no necesita crear primero un objeto `HolaMinisterio`.
- **void**: no devuelve un valor a la JVM.
- **main**: nombre del punto de entrada convencional.
- **String[] args**: argumentos recibidos desde la línea de comandos.

Cuando más adelante ejecutemos:

```
java ... HolaMinisterio Ana
```

`args[0]` contendrá "Ana".

System.out.println(...)

- **System**: clase de la biblioteca estándar.
- **out**: flujo de salida estándar.
- **println**: imprime y añade un salto de línea.
- el texto entre comillas dobles es un `String` literal.
- **;** finaliza la sentencia.

Errores frecuentes:

```
system.out.println("..."); // System mal escrito
System.out.println('texto'); // comillas simples no representan un String
System.out.println("texto") // falta ;
```

Las dos llaves } Una cierra `main`; la otra cierra la clase.

Pregunta

¿Por qué estamos haciendo este ejercicio sin Spring Boot?

Respuesta razonada

Porque queremos aislar la plataforma Java. Si este programa no compila o no se ejecuta, todavía no tiene sentido depurar Spring, Maven Web o un controlador. La prueba reduce el espacio de posibles causas.

Paso 7 - Compilar el programa

Desde la carpeta HolaMinisterio:

```
javac src/HolaMinisterio.java
```

Si todo va bien, javac puede no mostrar ningún mensaje. Comprueba que existe:

```
src/HolaMinisterio.class
```

Linux/macOS:

```
ls src
```

Windows:

```
dir src
```

Qué ha ocurrido

El compilador ha:

1. leído el código fuente;
2. comprobado sintaxis;
3. comprobado tipos;
4. generado bytecode.

Errores frecuentes

Mensaje aproximado	Causa probable
file not found	ruta o nombre equivocado
';' expected	falta ;
cannot find symbol	identificador mal escrito/no disponible
unclosed string literal	falta cerrar una cadena
reached end of file while parsing	llaves desequilibradas
class ... is public, should be declared...	fichero y clase pública no coinciden

Pregunta

¿Por qué una compilación correcta puede no imprimir absolutamente nada?

Respuesta razonada

Porque la finalidad normal de javac es producir clases, no un informe de éxito. La ausencia de errores y la aparición del .class son las evidencias. En herramientas de línea de comandos es común que el caso correcto sea silencioso.

Paso 8 - Ejecutar el programa

Ejecuta:

```
java -cp src HolaMinisterio
```

Resultado esperado:

Hola, Ministerio de Educación

Desglose

- java: arranca la JVM.
- -cp src: define el classpath y le indica dónde buscar clases.
- HolaMinisterio: nombre binario de la clase, sin .class.

Errores frecuentes

Could not find or load main class: revisa classpath, nombre y existencia del .class.

Main method not found: revisa la firma exacta de main.

Error de versión de clase: comprueba que la JVM no sea más antigua que el bytecode generado.

Pregunta

¿Qué papel cumple -cp src?

Respuesta razonada

Define dónde buscar clases. Como javac ha dejado HolaMinisterio.class dentro de src, la JVM necesita que esa carpeta forme parte del classpath para localizar la clase solicitada.

Paso 9 - Confirmar nuevamente Maven

Ejecuta:

```
mvn -version
```

Ahora ya puedes interpretar el resultado con más criterio: versión de Maven, Java usada por Maven y ruta de Java.

Si mvn sigue sin existir, anótalo, pero no bloquee el curso: el proyecto incluirá Wrapper.

Paso 10 - Configurar el IDE

El objetivo de este paso no es personalizar colores ni instalar decenas de plugins. Queremos demostrar que el IDE utiliza una JDK compatible y entiende el proyecto con las mismas reglas que la terminal.

IntelliJ IDEA

1. Abre IntelliJ IDEA.
2. En File > Project Structure, revisa Project SDK y selecciona JDK 17.
3. Comprueba el nivel de lenguaje del proyecto.
4. Abre la configuración de Maven y revisa qué JDK utiliza el importer/runner si tu versión muestra esa opción.
5. Si acabas de modificar el POM, recarga el modelo Maven.
6. Abre la terminal integrada y ejecuta `java -version` y `mvn -version`.

Qué debes observar: el proyecto no muestra errores de SDK y la JDK seleccionada es coherente con la baseline del curso.

Eclipse

1. Abre Window > Preferences > Java > Installed JREs.
2. Comprueba que JDK 17 está registrada; si no, usa Add > Standard VM y selecciona la raíz de la JDK.
3. Revisa Java > Compiler y el nivel de compatibilidad del proyecto.
4. Revisa la integración Maven/m2e.
5. Si el modelo está desactualizado, ejecuta Maven > Update Project.
6. Desde la terminal integrada o externa, vuelve a ejecutar `java -version` y `mvn -version`.

Qué debes observar: Eclipse resuelve el proyecto Maven y no intenta compilarlo con un JRE/JDK antiguo.

VS Code

1. Instala Extension Pack for Java.
2. Instala Spring Boot Extension Pack.
3. Ejecuta Java: Configure Java Runtime desde la paleta de comandos.
4. Comprueba que la JDK del proyecto es 17.
5. Si el language server conserva un modelo obsoleto, ejecuta Java: Clean Java Language Server Workspace y vuelve a abrir la carpeta.
6. En la terminal integrada, ejecuta `java -version` y `mvn -version`.

Qué debes observar: Java Language Server reconoce el proyecto y las extensiones no muestran errores causados por una JDK incompatible.

Verificación cruzada

Que el IDE diga «JDK 17» y que su terminal integrada muestre Java 11 son dos evidencias incompatibles. No lo ignores. Puede ocurrir porque:

- el IDE tenga una JDK configurada internamente;
- la terminal integrada herede un PATH antiguo;
- Maven use JAVA_HOME mientras el shell resuelve otro java;
- el IDE se abrió antes de cambiar las variables de entorno.

Si hay discrepancias, anota estas cuatro evidencias:

JDK del proyecto en el IDE
`java -version` en terminal integrada
`mvn -version` en terminal integrada
 ruta física de java/JAVA_HOME

Corrige una sola causa y repite las mismas comprobaciones.

Errores frecuentes

Problema	Causa probable	Acción
clases Java básicas en rojo	SDK no configurado	seleccionar JDK 17
dependencias Spring en rojo	modelo Maven no sincronizado	recargar Maven
compila en terminal pero no en IDE	SDK/modelo distinto	comparar configuración
el IDE sigue viendo Java antigua	proceso abierto antes del cambio	reiniciar IDE/terminal
VS Code no resuelve imports	Java Language Server obsoleto	limpiar workspace Java

Pregunta

¿Por qué un proyecto podría funcionar en terminal pero no desde el botón Run del IDE?

Respuesta razonada

Porque el botón Run puede utilizar otra JDK, otra configuración de classpath o un modelo Maven desactualizado. La terminal y el IDE son clientes distintos de las mismas herramientas. Si difieren, debemos comparar qué JDK y qué modelo de proyecto utiliza cada uno, no asumir que el código está mal.

Paso 11 - Tabla de diagnóstico del entorno

Síntoma	Causa probable	Acción
java no encontrado	PATH no contiene Java	corregir PATH
javac no encontrado	JDK no accesible	instalar/configurar JDK completa
mvn no encontrado	Maven global ausente	instalarlo o usar Wrapper
java = 11 y javac = 17	instalaciones mezcladas	unificar rutas
Maven usa Java antiguo	JAVA_HOME/configuración Maven	corregir selección de JDK
JAVA_HOME termina en bin	ruta incorrecta	apuntar a raíz JDK
IDE marca clases inexistentes	Maven no sincronizado	recargar proyecto
IDE usa otro SDK	configuración local	seleccionar JDK correcta

Paso 12 - Reto resuelto: argumento de línea de comandos

Sustituye el contenido por:

```
public class HolaMinisterio {
    public static void main(String[] args) {
        if (args.length > 0) {
            System.out.println("Hola, " + args[0]
                               + ", bienvenido al Ministerio de Educación");
        } else {
            System.out.println("Hola, Ministerio de Educación");
        }
    }
}
```

Qué hemos añadido

`args.length > 0` comprueba que exista al menos un argumento antes de acceder a `args[0]`. Sin esa condición, ejecutar sin argumentos provocaría un acceso fuera de los límites del array.

Recompila:

```
javac src/HolaMinisterio.java
```

Ejecuta con argumento:

```
java -cp src HolaMinisterio Ana
```

Resultado:

```
Hola, Ana, bienvenido al Ministerio de Educación
```

Ejecuta sin argumento:

```
java -cp src HolaMinisterio
```

Resultado:

```
Hola, Ministerio de Educación
```

Pregunta

¿Qué ocurre si ejecutas `java -cp src HolaMinisterio Ana Maria`?

Respuesta razonada

El shell entrega normalmente dos argumentos: `args[0] = "Ana"` y `args[1] = "Maria"`. Nuestro programa utiliza sólo `args[0]`, por lo que saludaría a Ana. Si quisiéramos tratar el nombre completo como un argumento habría que citarlo según el shell, por ejemplo "Ana Maria", o procesar todos los elementos.

Resultado esperado de la práctica 0.1

Debes haber demostrado que:

- el runtime funciona;
 - el compilador funciona;
 - las versiones son coherentes;
 - sabes interpretar Maven;
 - sabes localizar `JAVA_HOME` y `PATH`;
 - puedes compilar y ejecutar Java sin Spring;
 - puedes distinguir un problema de entorno de un problema de aplicación.
-

Práctica 0.2 - Crear el proyecto Spring Boot

Paso 1 - Abrir Spring Initializr

Abre:

<https://start.spring.io>

No descargues todavía. Primero identifica cada decisión que vas a tomar.

Pregunta

¿Por qué no empezamos simplemente descargando cualquier configuración que aparezca por defecto?

Respuesta razonada

Porque la versión por defecto y otras opciones pueden cambiar con el tiempo. Para que el curso sea reproducible necesitamos elegir conscientemente el sistema de build, la versión de Spring Boot, Java y las coordenadas.

Paso 2 - Configurar Project, Language y Spring Boot

Selecciona:

Project: Maven
Language: Java
Spring Boot: 3.5.16

Si Initializr ya no muestra esa versión en su interfaz principal, puedes utilizar el proyecto incluido en esta entrega, cuyo POM fija la baseline del curso. Lo importante es no sustituir silenciosamente la versión por una milestone, snapshot o major distinta.

Valores exactos de referencia para el curso

Configura:

Project: Maven
Language: Java
Spring Boot: 3.5.16
Group: es.mecd.demo

```
Artifact:    mi-proyecto
Name:        mi-proyecto
Description: Proyecto acumulativo del curso Spring Boot
Package name: es.mecd.demo.miproyecto
Packaging:   Jar
Java:        17
```

Añade:

Spring Web

El test starter llegará como parte de la plantilla Spring Boot generada.

Si Initializr cambia ligeramente el aspecto de la página o presenta una versión patch posterior dentro de la misma línea compatible, no copies la interfaz visual de memoria: verifica siempre los valores técnicos resultantes en pom.xml.

Por qué estos nombres importan

El group y el package name condicionan los packages Java.

El artifact participa en el nombre del JAR.

La versión Spring Boot fija el conjunto de dependencias gestionadas.

Java 17 fija el nivel mínimo de compilación que verificaremos en CI.

Pregunta

¿Por qué el curso no utiliza automáticamente la mayor más reciente sólo porque sea más nueva?

Respuesta razonada

Porque el objetivo es que los módulos 0-7 formen un recorrido coherente. Cambiar de mayor puede modificar APIs, servidor embebido, dependencias y comportamiento. Aprenderemos mejor sobre una baseline consistente y podremos estudiar una migración posteriormente como una operación explícita.

Paso 3 - Rellenar Project Metadata

Utiliza:

```
Group:        es.mecd.demo
Artifact:      mi-proyecto
Name:          mi-proyecto
Description:    Proyecto acumulativo del curso Spring Boot 2026
Package:        es.mecd.demo.miproyecto
Packaging:     Jar
Java:          17
```

Qué significa cada campo

Group. Espacio de nombres del artefacto.

Artifact. Identidad del artefacto dentro del group; también influye en el nombre del JAR.

Package. Raíz de paquetes Java.

Packaging JAR. Generaremos una aplicación ejecutable con servidor embebido.

Java 17. Nivel de lenguaje y baseline del curso.

Pregunta

¿Por qué groupId y paquete Java se parecen pero no son exactamente el mismo concepto?

Respuesta razonada

El groupId pertenece al modelo de coordenadas Maven. El package organiza tipos Java y condiciona visibilidad/escaneo. Es común derivar ambos del mismo dominio organizativo, pero Maven y Java los utilizan para fines distintos.

Paso 4 - Añadir dependencias

Añade sólo:

Spring Web

El proyecto generado incluirá además las dependencias de test habituales.

No añadas JPA, Security, Validation ni bases de datos todavía. Cada una tendrá su momento.

Pregunta

¿Qué ventaja pedagógica tiene empezar sólo con Spring Web?

Respuesta razonada

Cuando aparezca un nuevo comportamiento podremos relacionarlo con una dependencia que acabamos de introducir. Si cargáramos todo el ecosistema desde M0 perderíamos esa relación causa-efecto y Spring auto-configuraría piezas que aún no sabemos interpretar.

Paso 5 - Generar y descargar

Pulsa **Generate**. Obtendrás un ZIP.

Guárdalo en una carpeta de trabajo y conserva el ZIP hasta comprobar que la extracción está completa.

Paso 6 - Descomprimir y reconocer la raíz

Después de extraerlo, la carpeta raíz debe contener pom.xml.

Comprueba además que existen:

```
mvnw
mvnw.cmd
.mvn/
src/
```

La carpeta que contiene pom.xml es la que debes abrir como proyecto.

Error común

Abrir el directorio padre o abrir sólo src/ hace que el IDE no vea el proyecto Maven completo.

Paso 7 - Abrir el proyecto en el IDE

IntelliJ

File > Open y selecciona la carpeta del pom.xml. Espera a que Maven termine de importar.

Eclipse

File > Import > Maven > Existing Maven Projects, selecciona la raíz y termina la importación.

VS Code

File > Open Folder, selecciona la raíz. Espera al Java Language Server y a la importación Maven.

IntelliJ IDEA: importación detallada

1. File > Open.
2. Selecciona la carpeta que contiene pom.xml, no el ZIP ni la carpeta padre.
3. Si pregunta si debe confiar en el proyecto, revisa la ruta y acepta si es tu proyecto.
4. Espera a que Maven termine la sincronización.
5. Abre la ventana Maven y comprueba que aparece mi-proyecto.
6. Revisa el SDK del proyecto: Java 17.

Si ves imports Spring en rojo durante unos segundos, espera a que termine la resolución. Si persisten, recarga Maven.

Eclipse: importación detallada

1. File > Import.
2. Maven > Existing Maven Projects.
3. Elige la carpeta raíz que contiene pom.xml.
4. Eclipse detectará el POM.
5. Finaliza la importación y espera a m2e.
6. Revisa Project > Properties > Java Build Path / configuración JRE si existe alguna incoherencia.

VS Code: importación detallada

1. Instala Extension Pack for Java.
2. Instala el soporte Spring Boot que utilizarás durante el curso.
3. File > Open Folder.
4. Abre la carpeta raíz.
5. Espera a que Java Language Server importe el proyecto.
6. Comprueba el runtime detectado con Java: Configure Java Runtime.

Verificación independiente del IDE

Desde la raíz ejecuta:

Linux/macOS:

```
./mvnw -version
```

Windows:

```
mvnw.cmd -version
```

El IDE puede ser distinto, pero este comando debe seguir describiendo un proyecto Maven ejecutable.

Paso 8 - Verificar JDK y modelo Maven

Antes de ejecutar:

1. comprueba JDK 17/compatible;
2. comprueba que el POM no aparece con errores;
3. espera a que las dependencias hayan sido descargadas;
4. abre MiProyectoApplication.java y confirma que los imports de Spring se resuelven.

Si `org.springframework...` aparece como inexistente, no empieces cambiando imports: primero verifica que Maven ha sincronizado.

Paso 9 - Ejecutar desde el IDE

Ejecuta `MiProyectoApplication.main`.

Busca en los logs evidencias equivalentes a:

```
Tomcat started on port 8080
Started MiProyectoApplication
```

La sintaxis exacta varía por versión.

Pregunta

¿Que aparezca el banner de Spring Boot demuestra por sí solo que el servidor ha terminado de arrancar?

Respuesta razonada

No. El banner aparece al principio del proceso. La aplicación todavía puede fallar después al crear beans o al enlazar el puerto. Debemos buscar la finalización correcta del contexto y del servidor.

Paso 10 - Probar la raíz

Abre:

`http://localhost:8080/`

En este momento todavía no hemos creado un endpoint /. Es razonable recibir un 404.

Pregunta

¿Por qué un 404 no es equivalente a “Spring Boot no funciona”?

Respuesta razonada

Porque un 404 significa que el servidor ha atendido la petición y no ha encontrado un manejador para esa ruta. Si Spring Boot no hubiera arrancado o nadie escuchara en 8080, el navegador mostraría un fallo de conexión, no una respuesta HTTP 404 generada por la aplicación.

Paso 11 - Diagnóstico de creación/importación

Síntoma	Causa	Acción
pom.xml rojo	importación Maven incompleta	recargar Maven
JDK no válida	SDK del IDE incorrecto	seleccionar JDK 17/compatible
8080 ocupado	otro proceso escucha	parar proceso o cambiar temporalmente puerto
dependencias no descargan	red/proxy/repositorio	revisar error Maven, no editar Java
main no arranca	SDK o configuración	comparar con <code>./mvnw spring-boot:run</code>
/ devuelve 404	no existe mapping /	comportamiento esperado

Paso 12 - Reto resuelto: añadir DevTools manualmente

Abre `pom.xml` y localiza `<dependencies>`.

Añade:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

Explicación

groupId identifica el grupo Spring Boot.

artifactId identifica DevTools.

runtime expresa que no necesitamos compilar nuestro código contra clases de DevTools.

optional evita propagar esta dependencia como requisito transitivo si nuestro proyecto se utilizara como dependencia de otro.

Guarda el POM y recarga Maven.

Comprueba desde terminal:

Linux/macOS:

```
./mvnw dependency:tree
```

Windows:

```
mvnw.cmd dependency:tree
```

Busca spring-boot-devtools.

Pregunta

¿Por qué añadimos la dependencia editando el POM en vez de volver a generar el proyecto completo?

Respuesta razonada

Porque un proyecto Maven está diseñado para evolucionar. El POM es la declaración del build; añadir una dependencia es una modificación normal. Regenerar el proyecto podría sobrescribir trabajo y no enseña cómo se mantienen proyectos reales.

Variante adicional - Crear el mismo proyecto desde IntelliJ, Eclipse, VS Code y consola

La web de Spring Initializr es la referencia principal del curso porque expone los campos con claridad. Aun así, debes saber crear **el mismo proyecto** desde tu entorno habitual. El resultado debe conservar las mismas decisiones técnicas: Maven, Java, Spring Boot, coordenadas, paquete, JAR y Spring Web.

IntelliJ IDEA

1. Abre File > New > Project.
2. Selecciona el asistente de Spring/Initializr disponible en tu edición.
3. Elige Java y Maven.
4. Configura JDK 17.
5. Introduce es.mecd.demo como Group y mi-proyecto como Artifact.
6. Usa es.mecd.demo.miproyecto como paquete.
7. Selecciona JAR.
8. Añade **Spring Web**.
9. Finaliza la creación.
10. Abre pom.xml y comprueba que las decisiones coinciden con la baseline del curso.

Si la edición de IntelliJ que utilizas no ofrece el asistente de Spring, genera el ZIP desde la web y ábrelo con File > Open. La ausencia del asistente no cambia el proyecto Maven.

Eclipse + Spring Tools

1. File > New > Spring Starter Project.
2. Selecciona Maven y Java.
3. Comprueba JDK 17.
4. Introduce nombre, group, artifact y package.
5. Elige JAR.
6. En la pantalla de dependencias selecciona **Spring Web**.
7. Finaliza el asistente.
8. Espera la importación Maven.
9. Abre pom.xml y verifica las coordenadas y la dependencia.

Si Spring Starter Project no aparece, comprueba que Spring Tools esté instalado o utiliza la web y Import > Maven > Existing Maven Projects.

VS Code

1. Abre la Command Palette.
2. Ejecuta Spring Initializr: Create a Maven Project.
3. Selecciona la línea Spring Boot indicada por el curso si está disponible.
4. Elige Java.
5. Introduce es.mecd.demo como Group.
6. Introduce mi-proyecto como Artifact.
7. Selecciona JAR y Java 17.
8. Añade Spring Web.
9. Elige una carpeta donde generar el proyecto.
10. Abre la carpeta generada y espera la importación del Java Language Server.

La secuencia exacta de preguntas puede variar con la versión de la extensión. La verificación final siempre es el POM generado.

Consola mediante Spring Initializr

La API de Initializr permite automatizar la creación. Una forma práctica es descargar el ZIP con curl. Como los parámetros admitidos pueden evolucionar, consulta primero los metadatos del servicio si estás automatizando en un entorno real. El patrón conceptual es:

```
curl -G https://start.spring.io/starter.zip \
  --data-urlencode type=maven-project \
  --data-urlencode language=java \
  --data-urlencode groupId=es.mecd.demo \
  --data-urlencode artifactId=mi-proyecto \
  --data-urlencode name=mi-proyecto \
  --data-urlencode packageName=es.mecd.demo.miproyecto \
  --data-urlencode packaging=jar \
  --data-urlencode javaVersion=17 \
  --data-urlencode dependencies=web \
  -o mi-proyecto.zip
```

Después extrae el ZIP y abre la carpeta que contiene pom.xml.

Importante: para la edición final del curso el proyecto de referencia del repositorio fija el POM exacto. Este ejemplo de API enseña el mecanismo; si el servicio retira una versión histórica concreta de Spring Boot, no debes interpretar que el curso ha cambiado automáticamente de baseline.

Verificación común de las cuatro vías

Independientemente del canal, abre pom.xml y confirma:

```
groupId      = es.mecd.demo
artifactId   = mi-proyecto
java.version  = 17
packaging    = jar
Spring Web   = presente
```

Luego ejecuta desde la raíz:

```
./mvnw -version
./mvnw test
```

En Windows:

```
mvnw.cmd -version
mvnw.cmd test
```

Si los cuatro canales producen un proyecto equivalente, el aprendizaje importante es que **Initializr no pertenece a un IDE**. Los IDEs consumen el mismo servicio o importan el mismo resultado.

Pregunta

¿Qué archivo compararías primero para decidir si dos proyectos creados con canales distintos tienen realmente la misma configuración de build?

Respuesta razonada

pom.xml, porque describe coordenadas, parent, Java, dependencias y plugins Maven. Después compararía estructura de fuentes y configuración. Que las carpetas se vean distintas en el panel de un IDE no significa que el proyecto Maven sea diferente.

Resultado esperado de la práctica 0.2

Debes tener un proyecto Spring Boot 3.5.16 importado, capaz de arrancar y con Spring Web, DevTools y soporte de test declarados en el POM.

Práctica 0.3 - Entender la estructura y crear los primeros endpoints

Paso 1 - Abrir la raíz del proyecto

Confirma que estás en la carpeta que contiene:

```
pom.xml
mvnw
mvnw.cmd
src/
```

Paso 2 - Explorar la estructura completa

```
src/main/java
src/main/resources
src/test/java
```

Después ejecuta una construcción para observar target/:

Linux/macOS:

```
./mvnw package
```

Windows:

```
mvnw.cmd package
```

Ahora target/ existirá.

Recorre cada elemento y clasifícalo

No te limites a mirar el árbol. Para cada ruta, responde “fuente”, “recurso”, “test” o “generado”.

```
pom.xml
src/main/java/
src/main/resources/
src/test/java/
target/
.mvn/
mvnw
mvnw.cmd
```

pom.xml: definición Maven versionada.

src/main/java: código Java que formará parte de la aplicación.

src/main/resources: configuración y recursos que estarán en el classpath.

src/test/java: código de pruebas.

target: salida generada. Si todavía no existe, aparecerá al compilar.

.mvn + wrappers: infraestructura para ejecutar Maven de forma reproducible.

Experimento controlado con target

Ejecuta:

Linux/macOS:

```
./mvnw package
```

Windows:

```
mvnw.cmd package
```

Observa que aparece target/.

Después ejecuta:

```
./mvnw clean
```

o:

```
mvnw.cmd clean
```

target/ se elimina.

Tu código de src/ permanece intacto. Esa es la diferencia entre fuente y producto del build.

Inventario razonado de la raíz

Al comparar tu proyecto con el árbol de la guía pueden existir pequeñas diferencias según la versión de Initializr. Por ejemplo, algunas generaciones incluyen HELP.md y otras pueden no hacerlo. No conviertas una diferencia documental en un falso error de runtime.

Clasifica cada elemento en una de estas categorías:

Categoría	Ejemplos	¿Se versiona normalmente?
descriptor/build	pom.xml, .mvn/, mvnw, mvnw.cmd	
producción	src/main/java, src/main/resources	
tests	src/test/java	
documentación	README.md, HELP.md si existe	
artefactos generados	target/	
configuración local del IDE	depende del IDE/equipo	sólo si el equipo lo decide y aporta valor

Esta clasificación importa porque el repositorio debe contener lo necesario para reconstruir el proyecto, no los resultados generados por una máquina concreta.

Pregunta

¿Qué diferencia esencial hay entre src/ y target/?

Respuesta razonada

src/ contiene fuentes que mantenemos. target/ contiene productos generados por Maven: clases, informes y artefactos. Podemos borrar target/ y reconstruirlo; no deberíamos perder trabajo humano.

Paso 3 - Analizar pom.xml

No leas el POM como XML genérico: léelo por responsabilidades

Haz una segunda pasada y marca físicamente estas cinco zonas:

1. **parent**: de dónde hereda convenciones Spring Boot;
2. **coordenadas del proyecto**: groupId, artifactId, version;
3. **properties**: nivel Java y otras decisiones globales;
4. **dependencies**: capacidades que se incorporan;
5. **build/plugins**: herramientas que participan en la construcción.

Después ejecuta, sin modificar el archivo:

```
./mvnw dependency:tree
```

En Windows:

```
mvnw.cmd dependency:tree
```

Qué debes observar: Spring Web no es un único JAR. Bajo el starter aparecerán dependencias transitivas. El árbol ayuda a responder «¿por qué está esta librería en mi proyecto?» sin buscar manualmente en carpetas.

Como ejercicio de diagnóstico adicional puedes generar el POM efectivo:

```
./mvnw help:effective-pom
```

La salida es larga. No la copies en tu pom.xml. Úsala para comprobar que muchas versiones y configuraciones llegan por herencia/gestión aunque no estén escritas explícitamente en nuestro archivo.

Error frecuente: pensar que una dependencia sin <version> está incompleta. En un proyecto Spring Boot con su parent, muchas versiones están gestionadas deliberadamente.

Abre el fichero completo y localiza:

1. <parent>;
2. coordenadas del proyecto;
3. <properties>;
4. <dependencies>;

5. <build><plugins>.

El POM final de M0 es:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>

  <groupId>es.mecd.demo</groupId>
  <artifactId>mi-proyecto</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>mi-proyecto</name>
  <description>Proyecto acumulativo del curso Spring Boot 2026</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Pregunta

¿Por qué no aparece una versión explícita junto a spring-boot-starter-web?

Respuesta razonada

Porque el parent de Spring Boot aporta gestión de dependencias. El proyecto hereda un conjunto de versiones compatible. Añadir versiones manualmente sin necesidad puede romper esa coherencia.

Paso 4 - Analizar la clase principal

Abre:

src/main/java/es/mecd/demo/miproyecto/MiProyectoApplication.java

Debe ser:

```
package es.mecd.demo.miproyecto;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MiProyectoApplication {

    public static void main(String[] args) {
        SpringApplication.run(MiProyectoApplication.class, args);
    }
}
```

Qué debes entender

@SpringBootApplication es una anotación compuesta que activa la configuración principal de una aplicación Spring Boot, incluyendo auto-configuración y escaneo de componentes según sus convenciones.

SpringApplication.run(...) inicia el proceso de creación del contexto.

La ubicación del package es.mecd.demo.miproyecto es deliberada: los futuros paquetes estarán debajo.

Pregunta

¿Por qué no colocamos MiProyectoApplication dentro de ...miproyecto.app.inicio?

Respuesta razonada

Podríamos, pero reduciríamos el árbol de paquetes que se escanea de forma natural. Situarla en una raíz común simplifica el descubrimiento de controllers, services, repositories y configuración que añadiremos después.

Paso 5 - Analizar application.properties

Abre:

src/main/resources/application.properties

Contenido:

```
spring.application.name=mi-proyecto
```

El puerto no aparece porque estamos utilizando el valor por defecto 8080.

Para experimentar **temporalmente** podrías añadir:

```
server.port=8081
```

Arrancar, comprobar `http://localhost:8081`, parar y **eliminar después esa línea** para que el curso continúe en 8080.

Pregunta

¿Por qué indicamos explícitamente que este cambio de puerto es temporal?

Respuesta razonada

Porque los pasos posteriores y sus comandos esperan 8080. Un ejercicio puede alterar el estado para observar un concepto, pero debe dejar claro cuándo volver al estado base para no crear errores inexplicables más adelante.

Paso 6 - Crear el paquete controller

Dentro de:

```
src/main/java/es/mecd/demo/miproyecto/
```

crea:

```
controller/
```

El package resultante es:

```
package es.mecd.demo.miproyecto.controller;
```

Al estar debajo del package de la clase principal, Spring lo escaneará.

Paso 7 - Crear SaludoController

Crea:

```
src/main/java/es/mecd/demo/miproyecto/controller/SaludoController.java
```

con este contenido completo:

```
package es.mecd.demo.miproyecto.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class SaludoController {
```

```
    @GetMapping("/hola")  
    public String saludar() {  
        return construirMensaje("Ministerio de Educación");  
    }
```

```
    @GetMapping("/adios")  
    public String despedir() {  
        return "Adiós, Ministerio de Educación";  
    }
```

```
    private String construirMensaje(String destinatario) {  
        return "Hola, " + destinatario;  
    }
```

```
}  
}
```

Lee el archivo de arriba abajo

En este punto todavía no estamos diseñando una arquitectura por capas. El objetivo es comprender el camino mínimo:

```
HTTP GET /hola  
-> Spring MVC localiza @GetMapping  
-> invoca saludar()  
-> devuelve String  
-> Spring escribe la respuesta HTTP
```

Qué hace @RestController

Indica a Spring que esta clase participa en la capa web y que el valor devuelto por sus métodos se escribe en la respuesta, en lugar de interpretarse como nombre de una vista.

Qué hace @GetMapping

Asocia una petición HTTP GET con una ruta.

```
@GetMapping("/hola")
```

no significa “ejecutar este método al arrancar”. El método se ejecuta cuando llega una petición GET compatible.

Qué no hemos añadido todavía

No hay servicio, repositorio, DTO, validación ni base de datos.

Eso es deliberado. Si añadimos conceptos antes de enseñarlos, el proyecto deja de ser trazable pedagógicamente.

Explicación completa

@RestController hace que Spring registre esta clase como componente web y trate el valor de retorno de sus métodos de handler como cuerpo de respuesta.

@GetMapping("/hola") registra un mapping GET.

saludar() no recibe argumentos todavía. En módulos posteriores aprenderemos path variables, query parameters, cabeceras y cuerpos JSON.

construirMensaje(...) es un método Java privado. No es un endpoint. Spring no lo invoca directamente. Lo añadimos porque separa una pequeña operación y nos permitirá practicar Step Into de manera real en 0.4.

despedir() demuestra que una misma clase puede registrar más de una ruta.

Errores frecuentes

Falta @RestController. La clase puede compilar, pero Spring no la registra como controlador.

Falta @GetMapping. La clase existe como bean, pero ese método no tiene ruta HTTP.

Package fuera del escaneo. El bean puede no descubrirse.

Import incorrecto. Debe ser org.springframework.web.bind.annotation....

Pregunta

¿Por qué construirMensaje no puede visitarse en el navegador?

Respuesta razonada

Porque ser método de una clase controller no convierte automáticamente todos sus métodos en endpoints. Sólo saludar y despedir tienen anotaciones de mapping. construirMensaje es una implementación interna normal de Java.

Paso 8 - Arrancar la aplicación

Desde terminal:

Linux/macOS:

```
./mvnw spring-boot:run
```

Windows:

```
mvnw.cmd spring-boot:run
```

Espera a la finalización del arranque.

Paso 9 - Probar /hola en navegador

Abre:

`http://localhost:8080/hola`

Resultado esperado:

Hola, Ministerio de Educación

Si obtienes 404, comprueba:

- clase bajo el package correcto;
- `@RestController`;
- `@GetMapping("/hola")`;
- aplicación reiniciada/recargada;
- estás usando el puerto correcto.

Paso 10 - Probar con curl

```
curl http://localhost:8080/hola
```

Después:

```
curl -i http://localhost:8080/hola
```

Observa que ahora aparecen status y cabeceras, además del cuerpo.

Prueba también:

```
curl -i http://localhost:8080/adios
```

Pregunta

¿Qué información adicional ves con `-i`?

Respuesta razonada

Ves las cabeceras de la respuesta y la línea de estado HTTP. Esto permite distinguir el cuerpo de la semántica HTTP que lo acompaña, algo esencial cuando más adelante trabajemos con 201, 204, 400, 404 y otros estados.

Paso 11 - Diagnóstico de estructura/endpoints

Síntoma	Posible causa	Comprobación
404 en /hola	mapping no registrado	package/anotaciones/logs
no compila GetMapping	dependencia/import	POM y Maven
8080 no abre	app no arrancó	final de logs
responde en 8081	quedó cambio temporal	revisar properties
DevTools no recarga	IDE/build no detecta cambios	recompilar/reiniciar
/hola bien, /adios 404	falta segundo mapping	revisar método

Paso 12 - Reto resuelto: consolidar /adios

Si has seguido el código completo del Paso 7, /adios ya está incorporado como estado final. El reto consiste en entender la modificación como evolución del controlador.

La pieza añadida es:

```
@GetMapping("/adios")
public String despedir() {
    return "Adiós, Ministerio de Educación";
}
```

Comprueba:

```
curl http://localhost:8080/adios
```

Resultado:

```
Adiós, Ministerio de Educación
```

Pregunta

¿Podríamos crear POST /adios con @PostMapping y mantener a la vez GET /adios?

Respuesta razonada

Sí. Método HTTP y ruta forman conjuntamente el mapping. Serían operaciones distintas. Que técnicamente pueda hacerse no significa que sea un buen diseño REST; ese criterio lo estudiaremos en el Módulo 1.

Variante adicional - Crear paquetes y clases en los cuatro entornos

En la Práctica 0.3 creamos controller y SaludoController. La operación es conceptualmente la misma en todos los entornos, pero conviene dominar cómo se realiza en cada uno.

Desde consola y editor de texto

Crea la ruta de paquete bajo src/main/java:

Linux/macOS:

```
mkdir -p src/main/java/es/mecd/demo/miproyecto/controller
```

PowerShell:

```
New-Item -ItemType Directory -Force src/main/java/es/mecd/demo/miproyecto/controller
```

Después crea SaludoController.java con tu editor. La primera línea debe coincidir con la ruta lógica:

```
package es.mecd.demo.miproyecto.controller;
```

Compila con Maven para comprobar que ruta, package, imports y sintaxis son válidos:

```
./mvnw test
```

IntelliJ IDEA

1. Expande src/main/java.
2. Clic derecho sobre es.mecd.demo.miproyecto.
3. New > Package.
4. Escribe controller.
5. Clic derecho sobre el nuevo paquete.
6. New > Java Class.
7. Escribe SaludoController.
8. Pega o escribe el código del paso.

IntelliJ genera la declaración package a partir de la ubicación. Aun así, compruébala: el IDE facilita el trabajo, pero el package sigue siendo Java real.

Eclipse

1. Expande src/main/java.
2. Clic derecho sobre el paquete raíz.
3. New > Package.
4. Crea es.mecd.demo.miproyecto.controller.
5. Clic derecho sobre el paquete.
6. New > Class.
7. Nombre: SaludoController.
8. Finaliza y escribe el código.

Eclipse también mantiene sincronía entre el package elegido y la declaración Java. Si mueves una clase, prefiere las funciones de refactorización a arrastrar archivos sin revisar el package.

VS Code

Puedes trabajar desde el Explorer o desde la terminal integrada.

Explorer: crea las carpetas bajo src/main/java/es/mecd/demo/miproyecto/ y después el archivo SaludoController.java.

Java Project view: según las extensiones instaladas, podrás crear clases/packages desde las acciones Java. Si no aparece esa opción, no es un problema: la estructura de carpetas y la declaración package son lo que Java necesita.

Después ejecuta:

```
./mvnw test
```

Por qué enseñamos también la consola

La creación gráfica de una clase puede ocultar la relación entre:

```
src/main/java/es/mecd/demo/miproyecto/controller/SaludoController.java
```

y:

```
package es.mecd.demo.miproyecto.controller;
```

Crear una vez la estructura manualmente ayuda a comprender que el IDE no inventa packages: representa una convención de directorios y nombres que Java y Maven conocen.

Pregunta

¿Un package Java y una carpeta son exactamente lo mismo?

Respuesta razonada

No conceptualmente: el package es parte del nombre lógico de una clase Java. En la estructura convencional de Maven, ese nombre se refleja en directorios bajo `src/main/java`, y las herramientas esperan esa correspondencia. El IDE nos ayuda a mantenerla, pero el concepto pertenece a Java, no al IDE.

Resultado esperado de la práctica 0.3

El proyecto contiene una clase principal, configuración, un controlador con dos endpoints y una estructura que Maven y Spring reconocen correctamente.

Práctica 0.4 - Ciclo de trabajo diario sin depender de módulos posteriores

Requisitos previos

Debes tener el proyecto del punto 0.3 con:

```
GET /hola  
GET /adios
```

y el método privado `construirMensaje(...)`.

A diferencia de ejemplos que dependen de DTO o validaciones todavía no estudiadas, toda esta práctica utiliza exclusivamente código construido en M0.

Paso 1 - Arrancar desde el IDE

IntelliJ IDEA

1. Abre `MiProyectoApplication.java`.
2. Pulsa el icono Run junto a main o utiliza la configuración de ejecución creada por el IDE.
3. Observa la ventana Run.

Eclipse

1. Abre `MiProyectoApplication.java`.
2. Ejecuta Run As > Java Application o la opción Spring Boot equivalente si está disponible.
3. Observa Console.

VS Code

1. Abre `MiProyectoApplication.java`.
2. Utiliza Run sobre main o el panel Spring Boot si lo tienes instalado.
3. Observa Terminal/Debug Console según el modo.

IntelliJ IDEA: ejecución y configuración

1. Abre `MiProyectoApplication.java`.
2. Localiza el icono verde junto a la clase o main.
3. Elige **Run**.
4. Abre la ventana **Run** si no aparece automáticamente.
5. Comprueba que el command line utiliza la JDK esperada.
6. Abre Run > Edit Configurations y observa, sin cambiar nada todavía:
 - clase principal;
 - módulo/classpath;

- JRE;
- argumentos de programa;
- variables de entorno.

Ese diálogo explica por qué dos configuraciones aparentemente iguales pueden ejecutar con JDK o argumentos diferentes.

Eclipse: ejecución y configuración

1. En **Package Explorer**, abre la clase principal.
2. Run As > Java Application.
3. Observa **Console**.
4. Abre Run > Run Configurations.
5. Localiza la configuración creada.
6. Revisa:
 - Main class;
 - proyecto;
 - JRE;
 - Arguments;
 - Environment.

VS Code: ejecución y configuración

1. Abre la clase principal.
2. Usa Run sobre el método main.
3. Observa la salida.
4. Abre **Run and Debug**.
5. Si el editor crea una configuración launch.json, entiende que es configuración del IDE, no parte del código Spring.
6. Comprueba el runtime desde Java: Configure Java Runtime.

Verificación funcional común

En los tres IDE, después de arrancar:

```
curl -i http://localhost:8080/hola
```

debe devolver 200 y el texto esperado.

El botón cambia. La aplicación no.

Qué debes anotar

- versión de Java indicada en el arranque;
- puerto;
- tiempo aproximado;
- línea que indica que la aplicación está iniciada.

Pregunta

¿Qué factores pueden hacer que el primer arranque tarde más que los siguientes?

Respuesta razonada

Entre otros, descarga inicial de dependencias, calentamiento del sistema de archivos, indexación del IDE, carga de clases y cachés. No debemos comparar tiempos sin controlar esas variables ni interpretar unos cientos de milisegundos como una propiedad esencial del código.

Paso 2 - Leer los logs con intención

No leas el log como una pared de texto. Busca primero:

1. inicio de la aplicación;
2. versión de Spring Boot;
3. perfil activo si se informa;
4. servidor web y puerto;
5. finalización del arranque;
6. cualquier WARN o ERROR relevante.

Ahora visita:

`http://localhost:8080/hola`

Es posible que, con logging normal, no veas una línea detallando cada petición.

Experimento temporal

Añade a `application.properties`:

```
logging.level.org.springframework.web=DEBUG
```

Reinicia y vuelve a llamar `/hola`.

Observa la diferencia. Después **elimina esta línea** y vuelve a dejar:

```
spring.application.name=mi-proyecto
```

Pregunta

¿Por qué no dejamos DEBUG global activado permanentemente?

Respuesta razonada

Porque genera mucha más información, aumenta ruido y puede tener coste. Un nivel detallado debe utilizarse con intención, idealmente acotado al paquete que estamos diagnosticando. El objetivo del logging es encontrar señal, no maximizar líneas.

Paso 3 - Parar desde el IDE

Utiliza Stop en tu IDE.

Comprueba:

- el proceso termina;
- dejan de aparecer logs;
- el botón Stop queda inactivo;
- `http://localhost:8080/hola` deja de responder.

Si la URL sigue respondiendo, puede existir otro proceso en el mismo puerto.

Linux/macOS:

```
lsof -i :8080
```

Windows:

```
netstat -ano | findstr :8080
```

Pregunta

Si cierras una ventana del IDE pero `/hola` sigue respondiendo, ¿qué conclusión puedes sacar?

Respuesta razonada

Que probablemente el proceso Java sigue vivo o existe otra instancia escuchando en ese puerto. Cerrar una ventana de interfaz no es una prueba suficiente de que el proceso haya terminado; hay que comprobar el proceso o el puerto.

Paso 4 - Arrancar desde terminal

Linux/macOS:

```
./mvnw spring-boot:run
```

Windows:

```
mvnw.cmd spring-boot:run
```

Prueba /hola desde otra terminal.

Para parar, utiliza Ctrl+C.

Pregunta

¿Qué ventaja tiene saber arrancar sin el IDE?

Respuesta razonada

Nos permite distinguir el proyecto de una configuración gráfica local y reproduce mejor cómo trabajará un pipeline o un servidor. También es una vía de diagnóstico: si Maven arranca correctamente y el IDE no, el problema probablemente está en la configuración del IDE.

Paso 5 - Poner un breakpoint

Abre SaludoController.java y coloca un breakpoint en:

```
@GetMapping("/hola")
public String saludar() {
    return construirMensaje("Ministerio de Educación"); // breakpoint
}
```

Arranca en **Debug** desde tu IDE.

En otra terminal ejecuta:

```
curl http://localhost:8080/hola
```

La petición quedará esperando mientras el hilo está pausado.

Colocar el breakpoint según el IDE

IntelliJ: haz clic en el margen izquierdo de la línea. Aparece un punto rojo. Arranca con **Debug**.

Eclipse: doble clic en el margen o usa Toggle Breakpoint. Arranca con **Debug As**.

VS Code: haz clic en el margen. Usa **Run and Debug** o Debug sobre main.

El breakpoint debe estar en una línea ejecutable. Ponerlo en una llave o declaración que no genere una instrucción utilizable puede hacer que el IDE lo mueva o lo marque como no verificado.

Qué ocurre cuando haces la petición

1. curl abre conexión.
2. Tomcat acepta la petición.
3. Spring MVC resuelve /hola.
4. El hilo entra en saludar().

5. El depurador suspende ese hilo.
6. curl sigue esperando.
7. Al pulsar Continue/Resume, el hilo termina y la respuesta vuelve al cliente.

Este experimento conecta tres perspectivas: cliente HTTP, servidor y depurador.

Qué observar

- clase actual: SaludoController;
- método: saludar;
- pila de llamadas;
- hilo HTTP que atiende la petición;
- línea actual.

No hay todavía un id ni un DTO: no hemos enseñado esos conceptos. La práctica observa exactamente el código que el alumno ya ha construido.

Pregunta

¿Por qué curl parece quedarse “colgado” cuando el breakpoint se activa?

Respuesta razonada

Porque la petición HTTP está siendo atendida por un hilo cuya ejecución hemos pausado. El cliente espera a que el servidor termine el procesamiento y envíe la respuesta. Al continuar la depuración, la respuesta llega y curl termina.

Paso 6 - Comparar Step Over y Step Into

Sitúate en:

```
return construirMensaje("Ministerio de Educación");
```

Primero repite la petición y prueba **Step Over**. El depurador ejecutará construirMensaje sin detenerse dentro, salvo otros breakpoints.

Vuelve a disparar /hola, detente otra vez y utiliza **Step Into**. Ahora debes entrar en:

```
private String construirMensaje(String destinatario) {  
    return "Hola, " + destinatario;  
}
```

Inspecciona:

```
destinatario = "Ministerio de Educación"
```

Evalúa, si tu IDE lo permite:

```
"Hola, " + destinatario
```

Después usa Step Out o Continue.

Pregunta

¿Qué información nueva obtienes con Step Into respecto de Step Over?

Respuesta razonada

Step Into muestra el interior del método llamado y su estado local. En este caso podemos comprobar el parámetro destinatario y la expresión que construye la respuesta. Step Over sólo nos confirma el efecto final desde el método llamador.

Paso 7 - Crear y ejecutar un test del controlador

Crea:

src/test/java/es/mecd/demo/miproyecto/controller/SaludoControllerTest.java

```
package es.mecd.demo.miproyecto.controller;

import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

class SaludoControllerTest {

    private final SaludoController controller = new SaludoController();

    @Test
    void saludarDebeDevolverElMensajeEsperado() {
        assertThat(controller.saludar())
            .isEqualTo("Hola, Ministerio de Educación");
    }

    @Test
    void despedirDebeDevolverElMensajeEsperado() {
        assertThat(controller.despedir())
            .isEqualTo("Adiós, Ministerio de Educación");
    }
}
```

Ejecutar la misma prueba en cada IDE

IntelliJ

- abre SaludoControllerTest;
- usa el triángulo junto a la clase para ejecutar todos;
- usa el triángulo junto a un método para ejecutar sólo uno;
- prueba Debug sobre un test para detenerte dentro del código productivo.

Eclipse

- clic derecho sobre la clase;
- Run As > JUnit Test;
- para depurar, Debug As > JUnit Test.

VS Code

- abre el panel **Testing** o usa los enlaces sobre el método;
- ejecuta la clase completa o un test individual;
- usa **Debug Test** cuando quieras recorrerlo con el depurador.

Qué debes aprender de un fallo

Un test fallido no es simplemente una "X roja". Lee:

- nombre del test;
- valor esperado;
- valor real;
- stack trace;
- primera línea de tu código relevante.

La salida del test es una herramienta de diagnóstico antes de abrir el depurador.

Por qué este test es deliberadamente simple

Instanciamos directamente el controlador. Todavía no estamos probando routing HTTP ni cargando Spring. Eso llegará cuando el curso enseñe herramientas de test web. En M0 sólo necesitamos aprender:

- qué es un test;
- dónde vive;
- cómo ejecutarlo;
- cómo leer un fallo;
- cómo depurarlo.

Ejecuta la clase desde el IDE.

Pregunta

¿Este test demuestra que GET /hola está correctamente registrado en Spring MVC?

Respuesta razonada

No. Demuestra el comportamiento Java de los métodos del controlador. No arranca Spring ni envía una petición HTTP. Es importante no atribuir a una prueba garantías que no proporciona. Más adelante utilizaremos tests específicos de la capa web.

Paso 8 - Ejecutar todos los tests desde terminal

Linux/macOS:

```
./mvnw test
```

Windows:

```
mvnw.cmd test
```

Debes ver el build correcto y todos los tests verdes.

Para ejecutar una clase concreta:

Linux/macOS:

```
./mvnw -Dtest=SaludoControllerTest test
```

Windows:

```
mvnw.cmd -Dtest=SaludoControllerTest test
```

Pregunta

¿Por qué es valioso que el test pueda ejecutarse sin abrir el IDE?

Respuesta razonada

Porque convierte la prueba en parte del build reproducible. Un compañero, un servidor de CI o una máquina de integración puede ejecutar el mismo comando y obtener una señal objetiva sin depender de nuestros botones, plugins o configuraciones locales.

Paso 9 - Probar la API con curl

Con la aplicación arrancada, empieza por la petición mínima:

```
curl http://localhost:8080/hola
```

Qué hace: envía un GET /hola y muestra principalmente el cuerpo de la respuesta.

Por qué se hace: queremos separar la prueba HTTP del navegador y observar la API desde una herramienta de terminal reproducible.

Ahora incluye las cabeceras de respuesta:

```
curl -i http://localhost:8080/hola
```

Después activa el modo detallado:

```
curl -v http://localhost:8080/hola
```

Compara las tres ejecuciones:

- la primera prioriza el cuerpo;
- -i añade las cabeceras de la respuesta;
- -v enseña además información de conexión y de la petición que envía el cliente.

No uses -v por costumbre. Úsalo cuando tengas una pregunta de diagnóstico que justifique el ruido adicional.

Prueba el segundo endpoint:

```
curl -i http://localhost:8080/adios
```

Y una ruta que no existe:

```
curl -i http://localhost:8080/no-existe
```

Construye una pequeña matriz de observación

Ejecuta estas variantes y anota qué información aporta cada una:

```
curl http://localhost:8080/hola
curl -i http://localhost:8080/hola
curl -v http://localhost:8080/hola
curl -s http://localhost:8080/hola
curl -o salida.txt http://localhost:8080/hola
```

Comprueba el contenido de salida.txt. Las dos últimas opciones muestran que cURL sirve también para automatización: -s reduce salida de progreso y -o guarda el cuerpo.

Windows PowerShell: si curl no se comporta como esperas, ejecuta:

Get-Command curl

Si necesitas garantizar el binario cURL, utiliza curl.exe.

Qué debes observar en una respuesta

No te quedes sólo con el cuerpo. Identifica:

- código de estado;
- Content-Type;
- cabeceras relevantes;
- cuerpo;
- diferencia entre un mapping existente y una ruta inexistente.

Prueba otro puerto sin modificar el código

Empaqueta el proyecto y arráncalo así:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar --server.port=8081
```

Después:

```
curl -i http://localhost:8081/hola
```

Esto demuestra que el puerto puede suministrarse como configuración de ejecución. No hemos tenido que recompilar el controlador.

Errores frecuentes

Síntoma	Causa posible	Qué comprobar
conexión rechazada 404 en /hola	aplicación parada o puerto incorrecto	logs y puerto
curl no es el binario esperado en PowerShell	mapping ausente / app distinta	controlador y URL
la petición queda esperando durante Debug	alias/resolución del shell	Get-Command curl
	hilo pausado en breakpoint	ventana del depurador

Pregunta

¿Qué diferencia hay entre el 200 de /hola y el 404 de /no-existe si ambos llegan al mismo servidor?

Respuesta razonada

El servidor y Spring reciben ambas peticiones. En /hola encuentran un mapping y ejecutan un handler; en /no-existe no encuentran un recurso o manejador aplicable y producen una respuesta de no encontrado. El código de estado describe el resultado de esa petición, no si «hay servidor».

Paso 10 - Enviar cabeceras y cookies para observar la petición

Nuestro controlador todavía no consume cabeceras ni cookies, pero curl -v permite observar que el cliente las envía.

Cabecera personalizada:

```
curl -v -H "X-Curso: Spring-Boot-2026" http://localhost:8080/hola
```

Cookie:

```
curl -v -b "preferencias=es-ES" http://localhost:8080/hola
```

Esto no cambia la respuesta porque el controlador no lee esos datos. Su finalidad aquí es distinguir **enviar información HTTP** de **haber escrito código que la procese**.

Pregunta

¿Qué diferencia conceptual hay entre -H y -b?

Respuesta razonada

-H permite añadir una cabecera arbitraria. -b está especializado en enviar cookies y construye la cabecera Cookie apropiada. Las cookies viajan mediante cabeceras HTTP, pero curl proporciona una opción específica para gestionarlas con más comodidad.

Paso 11 - Tabla de errores del ciclo diario

Diagnóstico antes de «probar cosas»

Cuando aparezca un problema, no cambies varias variables a la vez. Sigue una secuencia reproducible:

1. copia el mensaje de error exacto;
2. identifica si ocurre al **compilar**, **testear**, **arrancar** o **atender una petición**;
3. reproduce el problema con el comando más pequeño posible;
4. comprueba versión/ruta sólo de las herramientas implicadas;
5. cambia una cosa;
6. repite exactamente la misma prueba.

Ejemplos:

- Si `./mvnw test` falla al compilar, todavía no tiene sentido depurar una petición HTTP.
- Si el JAR arranca pero `/hola` devuelve 404, Java y Maven ya han superado una parte importante del diagnóstico; el foco pasa al mapeo web.
- Si el IDE falla pero `./mvnw test` funciona, el proyecto es probablemente correcto y debemos revisar el modelo/configuración del IDE.
- Si el puerto 8080 está ocupado, cambiar código Java del controlador no puede resolver la causa.

Esta disciplina parece lenta sólo la primera vez. En proyectos reales evita ciclos largos de cambios aleatorios que destruyen la evidencia del problema original.

Problema	Evidencia	Diagnóstico inicial
Puerto 8080 ocupado	fallo durante arranque	localizar proceso/puerto
Breakpoint no se activa	peticion no llega o no estás en Debug	verificar URL/modo/mapping
curl espera indefinidamente	hilo pausado o servidor bloqueado	mirar depurador/logs
test verde en IDE pero Maven falla	configuraciones diferentes	ejecutar y leer Maven
Maven no encuentra test	nombre/ruta/compilación	revisar <code>src/test/java</code>
JAR no aparece	package falló	revisar errores previos
404 sólo en una ruta	mapping ausente	revisar anotación/ruta
IDE no refleja cambio	build/indexación/DevTools	recompilar o reiniciar

Paso 12 - Reto resuelto: localizar un fallo con el depurador

Vamos a provocar un error **en código que ya conocemos**, sin introducir validación de módulos posteriores.

Estado temporal: introduce un defecto

Cambia temporalmente:

```
private String construirMensaje(String destinatario) {
    return "Hola, " + destinatario;
}
```

por:

```
private String construirMensaje(String destinatario) {
    return "Adiós, " + destinatario;
}
```

No cambies el test.

Ejecuta sólo el test afectado

```
./mvnw -Dtest=SaludoControllerTest test
```

El test `saludarDebeDevolverElMensajeEsperado` debe fallar mostrando una diferencia entre el valor esperado y el real.

Depura el test

1. coloca un breakpoint dentro de `construirMensaje`;
2. ejecuta el test en Debug desde el IDE;
3. observa `destinatario`;
4. evalúa la expresión que se devuelve;
5. identifica que la palabra incorrecta nace dentro del helper;
6. corrige el método a `"Hola, " + destinatario`;
7. ejecuta nuevamente todos los tests.

Estado final obligatorio

El método debe quedar:

```
private String construirMensaje(String destinatario) {  
    return "Hola, " + destinatario;  
}
```

Y:

```
./mvnw test
```

debe volver a pasar.

Pregunta

¿Qué ha aportado el depurador que no aportaba ya el mensaje de AssertJ?

Respuesta razonada

El mensaje del test nos dice **qué** resultado difiere. El depurador nos permite observar **cómo** se construye ese resultado: argumentos, flujo y expresión concreta. En este ejemplo el fallo es trivial; el valor del ejercicio está en aprender el procedimiento antes de aplicarlo a cadenas de llamadas más complejas.

Recorridos completos por entorno: consola, IntelliJ IDEA, Eclipse y VS Code

La práctica anterior ha explicado el ciclo diario con un recorrido común. A partir de aquí repetimos ese mismo ciclo desde cuatro entornos distintos porque el alumno debe poder trasladar lo aprendido a la herramienta que utilice en su puesto de trabajo. No son cuatro proyectos distintos: es **el mismo proyecto**, con el mismo pom.xml, las mismas clases, los mismos tests y los mismos endpoints. Lo que cambia es cómo se inicia cada acción y dónde se observa su resultado.

Regla de trabajo: si un comportamiento funciona en un IDE pero falla desde Maven Wrapper, o al revés, no supongas que el código es correcto o incorrecto por intuición. Compara JDK, argumentos, perfil, directorio de trabajo, variables de entorno y modelo Maven. Dos lanzadores diferentes pueden estar ejecutando el mismo código con configuraciones diferentes.

Recorrido A - Ciclo diario exclusivamente desde consola

Este recorrido demuestra que el proyecto no depende del IDE. Es especialmente importante para CI/CD, servidores, diagnóstico remoto y automatización.

A1. Sitúate en la raíz correcta La terminal debe estar en la carpeta que contiene pom.xml y los wrappers:

```
mi-proyecto/  
|-- pom.xml  
|-- mvnw  
|-- mvnw.cmd  
`-- src/
```

Compruébalo antes de ejecutar nada:

Linux/macOS:

```
pwd  
ls
```

Windows PowerShell:

Get-Location
Get-ChildItem

Windows CMD:

```
cd  
dir
```

Si ejecutas Maven desde otra carpeta, el error no significa que Maven esté roto: simplemente no encuentra el proyecto.

A2. Verifica Java y Maven en la misma consola Linux/macOS:

```
java -version  
javac -version  
./mvnw -version
```

Windows:

```
java -version  
javac -version  
mvnw.cmd -version
```

La salida del wrapper debe indicar una JVM compatible con Java 17. Si `java -version` y Maven muestran JDK distintas, investiga antes de continuar: el comando Maven es el que determina con qué Java se compilará el proyecto.

A3. Compila sin ejecutar tests Linux/macOS:

```
./mvnw compile
```

Windows:

```
mvnw.cmd compile
```

Este paso separa los errores de compilación de los errores de test. Si `compile` falla, todavía no tiene sentido diagnosticar HTTP ni breakpoints.

A4. Ejecuta la suite de tests

```
./mvnw test
```

En Windows:

```
mvnw.cmd test
```

Busca dos cosas: que Surefire ejecute los tests y que el build termine con `BUILD SUCCESS`. Si el build falla, revisa primero el primer error útil y luego el resumen.

A5. Arranca con el plugin de Spring Boot Linux/macOS:

```
./mvnw spring-boot:run
```

Windows:

```
mvnw.cmd spring-boot:run
```

No cierras esa consola. El proceso ocupa el primer plano y seguirá escribiendo logs.

A6. Lee el arranque antes de abrir el navegador Confirma una secuencia razonable:

1. aparece el banner;
2. se inicializa el contexto;
3. Tomcat anuncia el puerto;
4. aparece el mensaje de aplicación iniciada;
5. no existe un stack trace posterior que invalide el arranque.

El banner por sí solo no es prueba suficiente.

A7. Prueba HTTP desde una segunda consola

```
curl -i http://localhost:8080/hola
curl -i http://localhost:8080/adios
curl -i http://localhost:8080/no-existe
```

Compara 200 y 404. Ambos demuestran que hay servidor; lo que cambia es si Spring encuentra un manejador para la ruta.

Para observar la conversación completa:

```
curl -v http://localhost:8080/hola
```

En PowerShell, usa `curl.exe` si necesitas evitar cualquier alias o función con el mismo nombre.

A8. Para la aplicación de forma ordenada Vuelve a la consola donde se está ejecutando Spring Boot y pulsa:

Ctrl+C

Después comprueba desde la segunda consola que la conexión ya no funciona:

```
curl -i http://localhost:8080/hola
```

Una conexión rechazada en este momento es el resultado esperado.

A9. Empaqueta y ejecuta el JAR

```
./mvnw package
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar
```

Windows:

```
mvnw.cmd package
java -jar target\mi-proyecto-0.0.1-SNAPSHOT.jar
```

Prueba de nuevo los endpoints. Este camino ya no utiliza `spring-boot:run`: ejecuta el artefacto que distribuiríamos.

A10. Cambia el puerto por argumento

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar --server.port=8081
```

Y en otra consola:

```
curl -i http://localhost:8081/hola
```

La prueba demuestra que una propiedad puede externalizarse sin modificar el código ni recompilar.

A11. Diagnostica un puerto ocupado Si 8080 ya está ocupado, localiza el proceso.

Linux/macOS:

```
lsof -i :8080
```

Linux también puede usar:

```
ss -ltnp | grep 8080
```

Windows:

```
netstat -ano | findstr :8080
```

No mates procesos indiscriminadamente. Identifica primero el PID y confirma qué programa es.

A12. Ejecuta un único test

```
./mvnw -Dtest=SaludoControllerTest test
```

Y un método concreto:

```
./mvnw -Dtest=SaludoControllerTest#saludarDebeDevolverElMensajeEsperado test
```

Esto resulta útil cuando una suite grande falla y quieres reducir el ciclo de diagnóstico.

Pregunta: ¿Qué garantiza este recorrido que no garantiza trabajar sólo con botones del IDE?

Respuesta razonada: demuestra que el proyecto es reproducible con herramientas estándar y que build, tests, empaquetado y ejecución no dependen de metadatos locales del IDE. Es la aproximación más cercana a lo que hará un servidor de integración continua.

Recorrido B - Ciclo diario completo en IntelliJ IDEA

Las rutas de menú pueden variar ligeramente entre versiones, pero la lógica es estable. Utiliza IntelliJ Community o Ultimate con una JDK 17 configurada para el proyecto.

B1. Abrir e importar el proyecto

1. File > Open.
2. Selecciona la carpeta que contiene pom.xml.
3. Confirma la confianza en el proyecto si IntelliJ lo solicita.
4. Espera a que termine la sincronización Maven.
5. Abre la ventana **Maven** y comprueba que aparece el proyecto.
6. En File > Project Structure, verifica que el **Project SDK** apunta a JDK 17.

Si los imports de Spring permanecen en rojo después de la sincronización, usa **Reload All Maven Projects** antes de editar el código.

B2. Ejecutar MiProyectoApplication Abre MiProyectoApplication.java. Junto al método main aparecerá un triángulo verde.

1. Pulsa el triángulo.
2. Selecciona **Run 'MiProyectoApplication'**.
3. Observa la ventana **Run** en la parte inferior.
4. Busca el puerto de Tomcat y el mensaje Started.

IntelliJ crea una Run Configuration. Puedes abrirla desde Run > Edit Configurations para ver JDK, working directory, argumentos y variables de entorno.

B3. Probar desde la terminal integrada Abre View > Tool Windows > Terminal y ejecuta:

```
curl -i http://localhost:8080/hola
```

También compara:

```
./mvnw -version
```

La terminal integrada hereda un entorno de proceso; si IntelliJ se abrió antes de cambiar PATH o JAVA_HOME, puede conservar valores antiguos. Reiniciar IntelliJ puede ser necesario.

B4. Parar correctamente En la ventana **Run**, pulsa el botón cuadrado rojo **Stop**. Después ejecuta curl otra vez para comprobar que 8080 ha dejado de responder.

No cierres la ventana de IntelliJ como forma habitual de parar el servidor: perderías información sobre si el proceso terminó de forma ordenada.

B5. Crear un breakpoint Abre `SaludoController.java` y haz clic en el margen izquierdo de la línea:

```
return construirMensaje("Ministerio de Educación");
```

Aparece un círculo rojo. Ese indicador representa un breakpoint de línea.

B6. Arrancar en Debug Pulsa de nuevo el triángulo de la clase principal, pero selecciona **Debug 'MiProyectoApplication'**. Cuando Tomcat termine de iniciar, llama:

```
curl http://localhost:8080/hola
```

La petición quedará detenida al alcanzar el breakpoint. En la ventana **Debug** observa:

- hilo actual;
- call stack;
- variables locales;
- botones Step Over, Step Into, Step Out y Resume.

B7. Entrar en el helper Usa **Step Into** para entrar en `construirMensaje`. Comprueba el valor de destinatario. Luego utiliza **Step Out** para volver a saludar y **Resume** para terminar la petición.

Si la terminal parece quedarse esperando, recuerda que el servidor está deliberadamente pausado por el depurador.

B8. Evaluar una expresión Con la ejecución detenida, utiliza **Evaluate Expression** y evalúa:

```
"Hola, " + destinatario
```

No modifica necesariamente el programa; permite comprobar hipótesis sobre valores en ese instante.

B9. Ejecutar tests desde IntelliJ Abre `SaludoControllerTest.java`.

- Triángulo junto a la clase: ejecuta todos los tests de la clase.
- Triángulo junto a un método: ejecuta sólo ese test.
- Debug: ejecuta el test bajo depurador.

En la ventana de resultados, distingue test pasado, fallado y no ejecutado. Para un fallo, lee diferencia esperado/real y stack trace.

B10. Ejecutar Maven desde la ventana Maven En la herramienta **Maven**, navega por Lifecycle y ejecuta test o package. Esto debe producir el mismo build Maven que en terminal, aunque la interfaz sea gráfica.

Después compara con:

```
./mvnw test
```

Si uno funciona y otro falla, revisa la configuración de Maven/JDK que usa IntelliJ.

B11. Crear y revisar una configuración de ejecución En Run > Edit Configurations revisa:

- Main class;
- JRE/JDK;
- working directory;
- program arguments;
- VM options;
- environment variables.

Para probar otro puerto, añade como **Program arguments**:

```
--server.port=8081
```

Arranca y verifica con curl.

B12. Diagnóstico específico de IntelliJ

Síntoma	Comprobación
imports Spring rojos	recargar Maven y esperar indexación
Run usa otra Java	Project SDK y JRE de la Run Configuration
Maven Tool Window usa configuración distinta	Maven settings / JDK for importer
breakpoint gris o no se activa	modo Debug, clase realmente cargada y ruta ejecutada
cambios no se reflejan	recompilar, reiniciar proceso o revisar DevTools

Pregunta: si `./mvnw test` funciona en la terminal integrada pero IntelliJ marca errores de compilación, ¿qué sospechas primero?

Respuesta razonada: el código y el POM ya tienen una evidencia fuerte a favor. Revisaría el modelo importado por IntelliJ, su SDK y la sincronización Maven antes de modificar código que Maven ha demostrado que compila.

Recorrido C - Ciclo diario completo en Eclipse

Utiliza Eclipse IDE for Enterprise Java and Web Developers o una instalación equivalente con soporte Maven y Java.

C1. Importar como proyecto Maven

1. File > Import.
2. Maven > Existing Maven Projects.
3. Selecciona la carpeta que contiene pom.xml.
4. Comprueba que Eclipse detecta el POM.
5. Pulsa **Finish**.
6. Espera a que m2e resuelva dependencias.

Si el proyecto aparece con errores mientras Maven todavía trabaja, espera a que finalice la resolución antes de cambiar imports.

C2. Verificar la JDK de Eclipse Abre Window > Preferences > Java > Installed JREs. Debe existir una JDK 17 y estar seleccionada cuando corresponda.

También revisa Project > Properties > Java Build Path y el nivel de compilación si aparece una discrepancia.

El nombre histórico del menú habla de “JRE”, pero para desarrollar queremos una instalación completa de JDK.

C3. Forzar la actualización Maven cuando haga falta Clic derecho sobre el proyecto:

Maven > Update Project...

Marca el proyecto y confirma. Utiliza esta acción cuando hayas cambiado el POM y Eclipse no haya actualizado el classpath.

C4. Arrancar la aplicación Abre MiProyectoApplication.java.

- clic derecho;
- Run As > Java Application o Spring Boot App si Spring Tools lo ofrece.

Observa la vista **Console**. Busca las mismas evidencias de arranque que en terminal.

C5. Parar la aplicación En la vista **Console**, pulsa el botón **Terminate** (cuadrado rojo). Asegúrate de terminar el proceso correcto si tienes varias consolas abiertas.

Después prueba:

```
curl -i http://localhost:8080/hola
```

Debe fallar la conexión si no hay otro proceso en ese puerto.

C6. Crear breakpoint Abre `SaludoController.java` y haz doble clic en el margen izquierdo junto a la línea que quieres detener. También puedes utilizar `Run > Toggle Breakpoint`.

Verifica que aparece la marca visual del breakpoint.

C7. Arrancar en Debug Clic derecho sobre `MiProyectoApplication.java`:

`Debug As > Java Application o Spring Boot App`.

Cuando llames a `/hola`, Eclipse cambia a la perspectiva **Debug** o te propone hacerlo. Observa:

- `Debug/Threads`;
- `Variables`;
- `Breakpoints`;
- editor en la línea detenida.

C8. Recorrer el código Los comandos habituales son:

- `Step Into`;
- `Step Over`;
- `Step Return`;
- `Resume`.

Los atajos exactos pueden variar por `keymap/OS`; el menú y los botones muestran la acción semántica que debes aprender.

Entra en `construirMensaje`, inspecciona `destinatario` y vuelve al método llamador.

C9. Inspeccionar y evaluar expresiones En la vista **Variables** puedes desplegar valores. La vista **Expressions** permite añadir expresiones como:

```
"Hola, " + destinatario
```

Esto es útil para comprobar hipótesis sin añadir `System.out.println` temporales por todo el código.

C10. Ejecutar tests Clic derecho sobre `SaludoControllerTest.java`:

`Run As > JUnit Test`.

Para depurar:

`Debug As > JUnit Test`.

La vista `JUnit` muestra tests pasados y fallidos. Si falla uno, abre su `stack trace` y navega a la línea correspondiente.

C11. Ejecutar Maven desde Eclipse Clic derecho sobre el proyecto:

`Run As > Maven test`

o crea una configuración `Maven Build...` con goals:

```
test
```

Para empaquetar:

```
package
```

Aunque Eclipse lance Maven mediante su integración, sigue siendo importante ejecutar también el wrapper desde una terminal externa o integrada disponible en tu instalación.

C12. Diagnóstico específico de Eclipse

Síntoma	Acción
Maven dependencies no aparecen	Maven > Update Project
Java compliance incorrecto	Properties / Installed JREs
proyecto conserva errores antiguos	Project > Clean después de resolver la causa
breakpoint no se detiene	comprobar Debug As y ruta llamada
vista Console muestra otro proceso	seleccionar consola correcta
test falla sólo en Eclipse	ejecutar mvnw test y comparar JDK/configuración

Pregunta: ¿por qué Project > Clean no debería ser tu primera respuesta ante cualquier error?

Respuesta razonada: limpiar recompila estado generado, pero no corrige una JDK incorrecta, una dependencia ausente o un POM erróneo. Debe usarse después de identificar una causa relacionada con estado de build, no como sustituto del diagnóstico.

Recorrido D - Ciclo diario completo en VS Code

VS Code es un editor extensible. Para que actúe como entorno Java/Spring necesita las extensiones adecuadas y un Java Language Server correctamente inicializado.

D1. Preparar extensiones Instala al menos:

- **Extension Pack for Java;**
- **Spring Boot Extension Pack** o las extensiones Spring equivalentes que utilices.

Reinicia VS Code si una extensión lo solicita.

D2. Abrir la carpeta correcta Utiliza File > Open Folder y abre la carpeta que contiene pom.xml.

No abras sólo src/main/java: el Java Language Server necesita ver el proyecto Maven completo.

D3. Verificar el runtime Java Abre la Command Palette y ejecuta:

Java: Configure Java Runtime

Comprueba que JDK 17 está disponible para el proyecto. Después abre el terminal integrado y ejecuta:

```
java -version
./mvnw -version
```

Una vez más, compara lo que ve el editor con lo que ve la terminal.

D4. Esperar la importación Java/Maven Al abrir un proyecto por primera vez, VS Code puede tardar en importar dependencias e indexar. No interpretes inmediatamente los imports rojos como errores de código.

Si la importación queda en mal estado, la Command Palette incluye comandos de limpieza/reinicio del workspace Java. Úsalos sólo después de verificar que el POM y la red son correctos.

D5. Arrancar la aplicación Abre `MiProyectoApplication.java`. Sobre main suelen aparecer enlaces **Run | Debug**.

Pulsa **Run**. La salida puede aparecer en terminal, Debug Console o panel relacionado según la configuración/extensión.

Confirma puerto y mensaje de arranque.

D6. Parar el proceso Utiliza el botón Stop de la sesión activa o termina el proceso en el terminal si lo has arrancado allí. Verifica con `curl` que el puerto ha quedado libre.**D7. Crear breakpoint y depurar** Haz clic en el margen izquierdo de `SaludoController.java` para crear el breakpoint. Luego inicia con **Debug** sobre main o desde **Run and Debug**.

Llama a `/hola`. Cuando se detenga, el panel **Run and Debug** muestra:

- VARIABLES;
- WATCH;
- CALL STACK;
- BREAKPOINTS.

D8. Step Over, Step Into, Step Out y Continue Utiliza la barra flotante/controles del depurador. Entra en `construirMensaje`, inspecciona `destinatario` y continúa.

En **WATCH** puedes añadir:

`"Hola, " + destinatario`

D9. Ejecutar tests Abre el panel **Testing**. VS Code detectará los tests JUnit si el proyecto está importado correctamente.

- ejecuta todos los tests;
- ejecuta un test individual;
- utiliza **Debug Test** para depurarlo.

Si los tests no aparecen, verifica primero que `./mvnw test` funciona. Después investiga la extensión y el Java Language Server.

D10. Ejecutar Maven desde terminal integrada

```
./mvnw test
./mvnw package
```

En Windows:

```
mvnw.cmd test
mvnw.cmd package
```

El terminal integrado es especialmente útil en VS Code porque mantiene a la vista editor y build reproducible.

D11. Configuración de ejecución con launch.json VS Code puede generar configuración de depuración bajo `.vscode/launch.json`. No es necesario dominarla en M0, pero conviene entender que puede fijar argumentos, working directory o variables.

Si defines un argumento para otro puerto, verifica siempre que la configuración local del editor no se convierte en requisito oculto del proyecto. El proyecto debe seguir arrancando con Maven Wrapper sin esos metadatos.

D12. Diagnóstico específico de VS Code

Síntoma	Acción
imports Java/Spring no resueltos	esperar/importar proyecto, comprobar POM
tests no aparecen	comprobar Maven y Java Test Runner
Run usa otra JDK	Java: Configure Java Runtime
debugger no se detiene	sesión Debug activa y breakpoint habilitado
workspace parece corrupto	limpiar workspace Java tras comprobar configuración
terminal y editor ven Java distintas	revisar entorno del proceso VS Code

Pregunta: ¿por qué en VS Code es especialmente importante diferenciar “editor” de “entorno Java”?

Respuesta razonada: VS Code por sí mismo es un editor generalista. El soporte de proyecto Java, Maven, tests y Spring procede de extensiones y del Java Language Server. Si una de esas piezas no está inicializada, el código puede ser correcto y Maven puede funcionar aunque el editor todavía no lo represente correctamente.

Tabla de equivalencias rápida de los cuatro recorridos

Acción	Consola	IntelliJ IDEA	Eclipse
Abrir proyecto	cd a la raíz	File > Open	Import > Maven > I
Ver Java	java -version	Project SDK + terminal	Installed JREs + ter
Recargar Maven	nuevo build / wrapper	Reload Maven Projects	Maven > Update Pr
Arrancar	mvnw spring-boot:run	Run sobre main	Run As
Depurar	JDWP avanzado, no necesario en M0	Debug	Debug As
Breakpoint	no aplica visualmente	margen del editor	doble clic margen
Tests	mvnw test	Run test	Run As > JUnit Test
Package	mvnw package	Maven Lifecycle/package	Maven Build/packag
Parar	Ctrl+C	Stop	Terminate
Probar HTTP	curl	terminal integrada / navegador	terminal / navegad

Pregunta de integración

Si el proyecto compila, pasa tests y arranca desde consola, pero falla al pulsar Run en un IDE, ¿qué conclusión inicial es más razonable?

Respuesta razonada

La evidencia apunta primero a una diferencia de configuración del entorno de ejecución del IDE: JDK, argumentos, variables, directorio de trabajo o sincronización Maven. No demuestra que el IDE sea el culpable con certeza, pero sí evita empezar a modificar un código que ya ha superado un build reproducible fuera del IDE.

Empaquetado y verificación final del Módulo 0

Paso final A - Ejecutar la suite

Linux/macOS:

```
./mvnw test
```

Windows:

```
mvnw.cmd test
```

Todos los tests deben pasar.

Paso final B - Empaquetar

```
./mvnw package
```

Debe aparecer un JAR bajo target/.

Paso final C - Arrancar el JAR

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar
```

En otra terminal:

```
curl -i http://localhost:8080/hola  
curl -i http://localhost:8080/adios
```

Esto prueba algo distinto de spring-boot:run: el artefacto generado puede ejecutarse como aplicación empaquetada.

Resultado esperado global del Módulo 0

Al terminar debes poder demostrar, no sólo afirmar, que:

- sabes qué JDK está usando el sistema;
- dispones de compilador;
- entiendes cómo Maven selecciona Java;
- has compilado y ejecutado Java sin Spring;
- has creado/importado un proyecto Maven Spring Boot;
- entiendes las partes principales del POM;
- sabes qué contienen src/main, resources, src/test y target;
- tienes GET /hola y GET /adios funcionando;
- puedes leer el arranque y detectar un fallo de puerto;
- sabes parar el proceso;
- sabes poner un breakpoint y diferenciar Step Over de Step Into;
- puedes ejecutar tests desde IDE y Maven;
- puedes empaquetar y arrancar el JAR;
- puedes inspeccionar respuestas HTTP mediante curl.

Estado final de los ficheros que has construido

MiProyectoApplication.java

```
package es.mecd.demo.miproyecto;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class MiProyectoApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(MiProyectoApplication.class, args);  
    }  
}
```

SaludoController.java

```
package es.mecd.demo.miproyecto.controller;  
  
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
public class SaludoController {

    @GetMapping("/hola")
    public String saludar() {
        return construirMensaje("Ministerio de Educación");
    }

    @GetMapping("/adios")
    public String despedir() {
        return "Adiós, Ministerio de Educación";
    }

    private String construirMensaje(String destinatario) {
        return "Hola, " + destinatario;
    }
}
```

application.properties

```
spring.application.name=mi-proyecto
```

SaludoControllerTest.java

```
package es.mecd.demo.miproyecto.controller;

import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

class SaludoControllerTest {

    private final SaludoController controller = new SaludoController();

    @Test
    void saludarDebeDevolverElMensajeEsperado() {
        assertThat(controller.saludar())
            .isEqualTo("Hola, Ministerio de Educación");
    }

    @Test
    void despedirDebeDevolverElMensajeEsperado() {
        assertThat(controller.despedir())
            .isEqualTo("Adiós, Ministerio de Educación");
    }
}
```

Puente al Módulo 1

M0 termina con una aplicación real pero muy pequeña. Ya sabemos **hacerla funcionar** y observarla. El Módulo 1 empezará a preguntar **por qué funciona**: qué ha auto-configurado Spring Boot, cómo se relacionan cliente y servidor, qué significa realmente una petición HTTP, cómo entran JSON y Jackson y cómo diseñar una API REST con criterio.